ALLIANCE UNIVERSITY | E1CSA313 DESIGN PROJECT-1

Blockchain-Based Verifiable Academic Credential System

Review 2 - Implementation Report

What was built after Review 1: the blockchain layer, the backend, the database connection and the security controls

Team	Manu M P (2411021061545) Tarun Kumar S (2411021061552) Anilkumar U K (2411021061563) Pruthviraj D N (L2411021061688)
Mentor	Dr. Saravanan P
Repository	github.com/anilkumaruk/blockchain
Date	24 October 2026

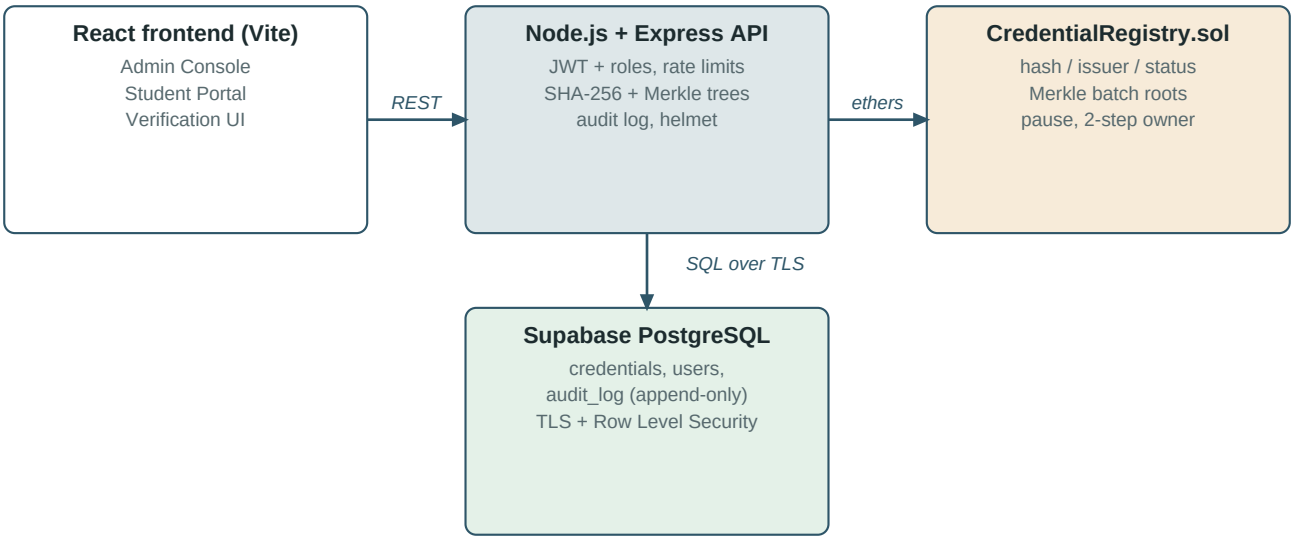
60	10 / 10	0.17%	23
automated tests passing	Review-1 scenarios automated	gas per credential in a 500-batch	security controls documented

1. Summary of changes

At Review 1 the prototype was a **frontend only**. The Admin Console, Student Portal and Verification UI worked, but every blockchain transaction and database call was simulated inside the browser (`src/lib/mockApi.js`). Only the SHA-256 hashing was real. For Review 2 the parts promised in the Review-1 methodology were built for real.

Area	At Review 1	Now (Review 2)
Blockchain	Simulated with timers and fake transaction hashes	Solidity **CredentialRegistry** contract (OpenZeppelin), deployed on a Hardhat Ethereum network; deploy script ready for Sepolia
Backend	None	Node.js + Express REST API, ethers.js blockchain bridge
Database	Browser localStorage	**PostgreSQL on Supabase** with versioned migrations, TLS and Row Level Security
Login	Any username and password accepted	Accounts in the database, scrypt-hashed passwords, JWT tokens, account lockout
Bulk issuance	Not available	**Merkle-tree batch issuance**: a whole class in one transaction
Audit	Not available	On-chain event history per credential and an append-only audit log
Testing	10 scenarios described on a slide	**60 automated tests**, including all 10 scenarios run end-to-end
Evaluation	Planned	Benchmark script measuring gas, latency, storage and detection accuracy

1.1 Architecture



The **blockchain is the source of truth** for each credential's fingerprint and status. The database holds the human-readable details. Every verification cross-checks the database record against the chain, so someone editing the database directly is detected as well.

Data	Stored in	Reason
SHA-256 of the certificate, issuer address, issue/revoke time, status	Blockchain	Tamper-evident and publicly verifiable
Merkle root of each batch	Blockchain	One 32-byte value covers hundreds of credentials
keccak256 of the revocation reason	Blockchain	Reason can be proven later without being published

Data	Stored in	Reason
Student name, ID, course, dates, reason text, Merkle proofs	Supabase PostgreSQL	Privacy: no personal data on-chain
User accounts (scrypt hashes) and audit log	Supabase PostgreSQL	Server-only access

2. Blockchain implementation

The smart contract `backend/contracts/CredentialRegistry.sol` (Solidity 0.8.24) builds on audited **OpenZeppelin** modules: `Ownable2Step`, `Pausable` and `MerkleProof`. It is compiled and tested with Hardhat, and the backend talks to it through **ethers.js v6**.

2.1 Contract functions

Function	Who can call	What it does
issueCredential(id, hash)	Authorized issuer	Stores the SHA-256 hash, issuer, timestamp and status *Valid*. Rejects duplicate IDs and reused hashes.
revokeCredential(id, reasonHash)	Original issuer	Changes the status to *Revoked*. Nothing is deleted, so the history stays auditable.
anchorBatch(root, count)	Authorized issuer	Stores one Merkle root covering a whole batch of credentials.
revokeBatchCredential(...)	Original issuer	Revokes one member of a batch using its Merkle proof, without affecting the others.
verifyCredential / getCredential / getCredentialByHash	Anyone (free)	Read-only checks by credential ID or by document hash. They cost no gas.
verifyBatchCredential(batchId, id, hash, proof)	Anyone (free)	Checks the Merkle proof against the anchored root.
setIssuer(address, name, active)	Owner	Authorizes or deactivates an issuing institution's address.
pause() / unpause()	Owner	Emergency stop for issuing and revoking. Verification keeps working.
transferOwnership / acceptOwnership	Owner / new owner	Two-step transfer: the new owner must accept, so a wrong address cannot take control.

Events `CredentialIssued`, `BatchAnchored`, `CredentialRevoked` and `IssuerUpdated` are emitted for every change. The API reads them to show the **on-chain audit trail** of a credential.

2.2 Merkle-tree batch issuance

Issuing credentials one at a time costs about 125,000 gas each. With batch issuance the backend hashes every certificate, builds a Merkle tree from the hashes and writes only the **root** on-chain in a single transaction. Each student's record keeps a short **proof** (4 to 9 hashes) that links their certificate to that root, and the contract checks the proof during verification. Tree leaves are hashed twice, which blocks a known attack where an internal tree node is passed off as a leaf (a second-preimage attack).

Batch size	Total gas	Gas per credential	Compared with individual	Proof length
1 (individual)	124,957	124,957	100%	-
10	122,934	12,293	9.84%	4 hashes
50	105,834	2,117	1.69%	6 hashes
100	105,834	1,058	0.85%	7 hashes
250	105,822	423	0.34%	8 hashes
500	105,834	212	0.17%	9 hashes

Measured with `npm run benchmark` on the Hardhat network.

2.3 Cost, storage and accuracy

Measure	Result
Issue one credential	~125,000 gas (~0.00125 ETH at 10 gwei on Ethereum mainnet; ~0.000006 ETH on a low-fee L2)
Revoke one credential	~44,500 gas
Verify	Free (read-only call)
Deploy the contract (one time)	1,638,249 gas
On-chain storage	128 bytes per individual credential; 96 bytes per batch of any size
Genuine certificates verified	50 / 50 (100%)
Certificates with a single changed byte detected	50 / 50 (100%)
Revoked credentials reported as revoked	10 / 10 (100%)

3. Backend API

The backend (`backend/server/`) is a Node.js + Express REST API. It computes the certificate hash itself rather than trusting the browser, sends transactions to the contract through ethers.js, and stores the readable details in PostgreSQL. Response formats match the old mock, so the React pages work unchanged. Setting `VITE_API_URL` switches the frontend from the mock to the real backend.

Method and path	Access	Purpose
POST /api/auth/login	Public (rate-limited)	Sign in and receive a JWT
POST /api/credentials	Admin	Hash the certificate, anchor it on-chain, save the record
POST /api/credentials/batch	Admin	Merkle batch: up to 500 credentials in one transaction
POST /api/credentials/:id/revoke	Admin	Revoke on-chain (status change, no deletion)
POST /api/credentials/:id/reissue	Admin	Revoke and issue a corrected credential, linked both ways
GET /api/credentials	Admin	List all credentials
GET /api/verify/:id	Public	Verify by credential ID or QR code
POST /api/verify/file	Public	Upload a PDF; the server recomputes SHA-256
POST /api/verify/hash	Public	Verify a SHA-256 computed in the browser
GET /api/credentials/:id/history	Public	On-chain event history
GET /api/students/:id/credentials	Public	Student portal and shared profile
GET /api/admin/audit	Admin	Audit log
GET /api/admin/registry, POST /api/admin/pause	Admin	Contract status and emergency pause
GET /api/health	Public	Blockchain and database status

Verification results: **valid**, **revoked**, **invalid** (with a reason code: not_found, hash_mismatch, untrusted_issuer or record_mismatch) or **unavailable** when the blockchain cannot be reached. The system never reports a false pass.

4. Database: Supabase PostgreSQL

The in-browser store was replaced by PostgreSQL hosted on **Supabase** (PostgreSQL 17, Tokyo region `ap-northeast-1`). The connection goes through Supabase's Session pooler, which works on IPv4 networks. The schema is created by versioned SQL migrations that run automatically when the server starts.

Table	Contents	Protection
credentials	Student and credential details, on-chain transaction info, batch proofs	Hash format check, unique hash, valid status values, links between corrected credentials
users	Usernames, roles, scrypt password hashes, failed-login counter, lock time	Passwords never stored in plain text
audit_log	Time, user, action, target, IP address, details	A database trigger blocks every UPDATE and DELETE (append-only)
schema_migrations	Which migrations have run	-

4.1 Supabase-specific security

- **Verified TLS:** the connection is encrypted and checked against Supabase's CA certificate (`certs/prod-ca-2021.crt`). The server refuses to connect to Supabase without it.
- **Row Level Security:** Supabase automatically exposes tables through a public REST API. Migration `002_lock_down_public_api.sql` enables RLS on every table and removes the `anon` and `authenticated` roles' access, so student data, password hashes and the audit log cannot be read with the project's public key.
- **Connection resilience:** keep-alive and early retirement of idle connections, so connections dropped by the pooler do not break requests.
- **Secrets kept out of git:** the connection string is in `backend/.env`, and both `.env` and the certificate folder are git-ignored.
- **Check command:** `npm run db:check` confirms the connection, TLS, migrations and RLS without printing any password.

5. Security controls

Each control below is covered by an automated test. The full threat model is in `SECURITY.md`.

Threat	Control
Forged or edited certificate	SHA-256 anchored on-chain; a single changed byte fails verification
Certificate from a fake university	Only this university's issuer address is trusted (untrusted_issuer)
Insider edits the database	Every verification compares the record with the on-chain hash (record_mismatch)
Revocation erased to hide history	Revocation is a status change; issue and revoke events stay on-chain
Unauthorized issuance through the API	JWT authentication plus role check on every admin route
Unauthorized issuance directly on-chain	onlyIssuer check; only the original issuer can revoke
Leaked password database	scrypt hashing with a unique random salt per password
Password guessing	Lock after 5 failed attempts for 15 minutes; 20 logins per 15 minutes per IP
Discovering valid usernames	Same error and same response time for unknown users and wrong passwords
Forged login tokens	HS256 only (rejects alg=none); issuer, audience and 1-hour expiry checked
Unsafe production setup	Server refuses to start without a strong JWT secret, a database, a CORS allowlist and a real issuer key
Stolen issuer key	Owner can pause the contract and deactivate the issuer

Threat	Control
Contract handed to a wrong address	Two-step ownership transfer (Ownable2Step)
Forged Merkle proof	OpenZeppelin MerkleProof with double-hashed leaves
SQL injection and oversized input	Parameterized SQL only; field validation; request size limits
Uploading non-PDF files	PDF signature check and a 5 MB limit
Browser attacks (clickjacking, sniffing)	Security headers via helmet (CSP, HSTS, X-Frame-Options)
Other websites calling the API	CORS allowlist
Request flooding	Rate limits on verification and admin actions
Tampering with the audit trail	Database trigger makes audit_log append-only
Public access to Supabase tables	Row Level Security plus revoked anon/authenticated access
Blockchain outage shows a false 'valid'	Fails safe: returns 503 unavailable, never a verdict
Half-saved batch	Batch records saved in one database transaction

6. Testing

Test file	Tests	Covers
CredentialRegistry.test.js	8	Contract: issue, verify, revoke, duplicates, permissions, issuer status
batch.test.js	17	Merkle tree vs. contract, batch proofs, batch revocation, gas saving, batch API
scenarios.test.js	11	The 10 Review-1 scenarios end-to-end, plus database tampering
security.test.js	18	Passwords, lockout, tokens, roles, headers, rate limits, input validation, audit log, pause, ownership, startup checks, Supabase TLS
postgres.test.js	6	Real PostgreSQL: migrations and RLS, issue/verify/revoke, batches, constraints, lockout, append-only audit log
Total	**60**	**All passing**

6.1 Review-1 experimentation scenarios (now automated)

#	Test case	Condition	Expected	Result
1	Valid credential	Original certificate	Verified	Pass
2	Modified certificate	One character changed in the PDF	Hash mismatch	Pass
3	Invalid credential ID	Unknown ID	Not found	Pass
4	Revoked credential	University revoked it	Revoked	Pass
5	Wrong issuer	Issued by an untrusted address	Verification failed	Pass
6	Duplicate credential ID	Existing ID reused	Rejected (API and contract)	Pass
7	Unauthorized issuance	Student or no token	Access denied	Pass
8	Blockchain unavailable	Network down	Unavailable (503)	Pass
9	Invalid file format	PNG or empty file	File rejected (415)	Pass
10	Corrected credential	Old revoked, new issued	New credential valid	Pass

7. Frontend changes

- `src/lib/api.js` switches between the real backend and the in-browser mock, so the public Vercel demo keeps working without a server.
- Admin sign-in now checks the username and password against the server and receives a JWT. Sign out discards the token.
- New **Batch Issue** tab: paste or upload a CSV and issue a whole class in one transaction.
- New **Security** tab: contract status, emergency pause button and the audit log.
- Verification results show the **on-chain audit trail** (issued, anchored, revoked) read from the blockchain.
- Server error messages (account locked, registry paused, blockchain unavailable) are shown to the user instead of a generic failure.

8. Files added and changed

File	Status	Purpose
backend/contracts/CredentialRegistry.sol	New	Solidity smart contract
backend/server/app.js	New	REST API routes, verification logic, rate limits, error handling
backend/server/chain.js	New	ethers.js bridge: transactions, reads, events, timeouts
backend/server/store.js	New	PostgreSQL data layer and migration runner (in-memory store for tests)
backend/server/auth.js	New	JWT login, role checks, account lockout
backend/server/passwords.js	New	scrypt password hashing
backend/server/merkle.js	New	Merkle tree and proofs compatible with OpenZeppelin
backend/server/config.js	New	Environment validation, production safety checks, Supabase TLS
backend/server/index.js	New	Server startup, first admin account
backend/server/migrations/001_init.sql	New	Tables, constraints, append-only audit trigger
backend/server/migrations/002_lock_down_public_api.sql	New	Row Level Security for Supabase
backend/scripts/deploy.js, seed.js	New	Deploy the contract; load the demo credentials
backend/scripts/benchmark.js	New	Performance evaluation; writes results/benchmark.md
backend/scripts/create-user.js, db-check.js	New	Create accounts; check the database connection
backend/test/*.test.js	New	60 automated tests
backend/hardhat.config.cjs, docker-compose.yml, .env.example	New	Blockchain, local database and configuration setup
src/lib/api.js, src/lib/httpApi.js	New	Frontend switch and backend client
src/components/BatchIssuePanel.jsx	New	Batch issuance screen
src/components/SecurityPanel.jsx	New	Contract status, pause and audit log screen
src/components/OnChainHistory.jsx	New	On-chain history in verification results
src/pages/AdminConsole.jsx	Changed	Real login, new tabs, server error messages
src/pages/VerificationUI.jsx	Changed	Error handling when verification cannot complete
src/components/VerificationResult.jsx	Changed	Shows on-chain history
src/components/PrototypeBanner.jsx	Changed	Shows whether the app uses the mock or the real server
src/pages/StudentPortal.jsx, PublicProfile.jsx	Changed	Use the new API switch
README.md, SECURITY.md	Changed / New	Setup guide, API reference, threat model

9. Current status and next steps

9.1 Open items

- **Admin sign-in on Supabase:** one sign-in attempt returned a server error. The database and the login code were both checked against Supabase and work correctly. The most likely cause, idle database connections

dropped by the pooler, has been fixed, but this still needs confirming after a server restart.

- **Contract network:** the contract runs on a local Hardhat Ethereum network. Restarting that network erases it, while Supabase keeps its data, so the two can fall out of step during development. Deploying to the **Sepolia** testnet (script ready) makes the contract permanent and publicly viewable.
- **Student login in the portal:** the backend supports student accounts, but the Student Portal page still shows the demo student.
- **Not yet built:** IPFS storage for certificate files and OCR/NLP content checking, both listed as future directions in Review 1.

9.2 Known limitations

- A single issuer key on the server; a hardware wallet or multisig owner would be stronger.
- Blockchain and database writes are not one atomic step. A failure between them can leave a credential on-chain without its database record. This cannot be used to fake a credential.
- Login tokens cannot be revoked before their 1-hour expiry.
- The contract has not had an external security audit.

9.3 How to run

```
cd backend
npm run db:check        # Supabase connection, TLS, RLS
npm run chain           # terminal 1: local blockchain
npm run deploy:local && npm start  # terminal 2: contract + API
npm run seed            # demo credentials
npm run test:db         # all 60 tests (needs Docker: npm run db:up)
npm run benchmark       # results/benchmark.md
cd .. && VITE_API_URL=http://localhost:4000 npm run dev
```